

HTML. Unidad 9. Formularios.



[1. Presentación](#)

[1.1. Infografía](#)

[1.2. Diapositivas y vídeo](#)

[2. Introducción](#)

[3. ¿Qué son los formularios web?](#)

[4. Campos de un formulario](#)

[4.1. El elemento <label>](#)

[4.1.1. ¡También se puede hacer clic en las etiquetas!](#)

[4.2. Campos de entrada de texto](#)

[4.2.1. Campos de texto de una sola línea](#)

[4.2.1.1. Campo de contraseña](#)

[4.2.2. Campos de texto de varias líneas](#)

[4.2.3. Valores por defecto en los cuadros de texto](#)

[4.3. Casillas de verificación y botones de opción](#)

[4.3.1. Casillas de verificación](#)

[4.3.2. Botones de opción](#)

[4.4. El elemento <select>](#)

[4.5. Selector de archivos](#)

[4.6. Botones](#)

[4.7. Ejercicio propuesto: Campos básicos](#)

[5. Campos nuevos en HTML5](#)

[5.1. Campo de dirección de correo electrónico](#)

[5.2. Campo URL](#)

[5.3. Campo número de teléfono](#)

[5.4. Campo numérico](#)

[5.5. Controles deslizantes](#)

[5.6. Selectores de fecha y hora](#)

[5.7. Selector de color](#)

[5.8. Campo de búsqueda](#)

[5.9. Ejercicio propuesto: Campos HTML5](#)

[6. Atributos comunes](#)

[7. Un primer formulario «real»](#)

[7.1. Diseñar el formulario](#)

[7.2. Implementando nuestro formulario](#)

[7.3. El elemento <form>](#)

[7.4. Los elementos <label>, <input> y <textarea>](#)

[7.5. El elemento <button>](#)

[7.6. Enviando datos al servidor web](#)

[7.6.1. Validación de formulario en la parte del cliente](#)

[7.6.1.1. A tener en cuenta...](#)

[7.6.2. En el lado del servidor: Recuperando los datos](#)

[7.6.3. El método GET](#)

[7.6.3.1. Ejercicio propuesto: Formulario de contactar](#)

[7.6.3.2. Ejercicio propuesto: Guardar datos de contacto](#)

[7.6.3.3. Ejercicio propuesto: Formulario para saludar](#)

[7.6.3.4. Ejercicio propuesto: Guardar texto genérico](#)

[7.6.4. El método POST](#)

[7.6.4.1. Ejercicio propuesto: Formulario de login](#)

[7.6.4.2. Ejercicio propuesto: Comprobar datos privados](#)

[8. Test](#)

1. Presentación

1.1. Infografía

Guía Esencial de Formularios HTML: De la Etiqueta al Servidor

1. ¿Qué es un Formulario Web?

El principal punto de interacción con el usuario

Permiten a los usuarios introducir datos que generalmente se envían a un servidor web para su procesamiento y almacenamiento

Compuesta por "Controles" o "Widgets"

Elementos como campos de texto, botonas y casillas de verificación que se utilizan para recopilar la información del usuario

2. La Estructura Fundamental

<form>

action="uri_destino" method="GET/POST"

Todo comienza con **<form>**. Este elemento contenedor define el formulario y sus dos atributos clave

La etiqueta <label> es crucial para la accesibilidad

Nombre:

Asocia un texto descriptivo a un control del formulario, ayudando a los lectores de pantalla y ampliando el área de clic para todos los usuarios

<label>Nombre: <input type="text" name="nombre">

Asociar etiquetas es fácil. Se puede hacer anidando el **<input>** dentro del **<label>** o vinculándolos con los atributos **for** e **id**

3. Controles de Formulario Clásicos

Texto largo

Password

Selección Múltiple vs. Única (checkbox vs. radio)

Botón JS

Controles Modernos con HTML5

email, url, tel Entradas con Validación y Teclados Específicos, validación integrada y teclados optimizados

date, time, color Selectores de Interfaz Nativos, abren los selectores nativos del sistema

4. Enviando los Datos: GET vs. POST

Método GET

.../buscar.php?q=html

Visibilidad de Datos: Los datos se añaden a la URL (públicos). Seguridad: Inseguro para datos sensibles

Ejemplo de URL: .../buscar.php?q=html

Caso de Uso Típico: Búsquedas, filtros, datos no sensibles

Método POST

.../login.php

Visibilidad de Datos: Los datos se envían en el cuerpo de la petición (ocultos). Seguridad: Más seguro. Recomendado para datos sensibles.

Ejemplo de URL: .../login.php

Caso de Uso Típico: Formularios de login, registro, envío de archivos

5. La Validación es Clave

Validación en el Lado del Cliente

Verificación inicial en el navegador que asegura que los datos cumplan el formato requerido

Atributos HTML5 para Validar sin JavaScript:

required, minlength, max, pattern

"¡No es una medida de seguridad exhaustiva!"

La validación en el cliente es fácil de eludir. Siempre debes realizar una validación de seguridad en el lado del servidor para proteger tu aplicación

NotebookLM

1.2. Diapositivas y vídeo

A través de esta [presentación](#), estas [diapositivas](#), y este [vídeo](#), te familiarizarás con los contenidos clave de esta unidad.

2. Introducción

Esta unidad proporciona instrucciones y ejemplos que te ayudarán a aprender los conceptos básicos de los formularios web, que son una de las principales herramientas para interactuar con los usuarios. Por lo general, se utilizan para recopilar datos o para permitirles controlar una interfaz de usuario. Sin embargo, por razones históricas y técnicas, no siempre es obvio cómo utilizarlos en todo su potencial. En las secciones que se

enumeran a continuación, cubriremos todos los aspectos esenciales de los formularios web, incluido el marcado de su estructura HTML, la validación de los datos del formulario y el envío de datos al servidor.

3. ¿Qué son los formularios web?

Los **formularios web** son uno de los principales puntos de interacción entre un usuario y un sitio web o aplicación. Los formularios permiten a los usuarios la introducción de datos, que generalmente se envían a un servidor web para su procesamiento y almacenamiento (puedes consultar [Enviar los datos de un formulario](#) para más detalles), o se usan en el lado del cliente para provocar de alguna manera una actualización inmediata de la interfaz (por ejemplo, se añade otro elemento a una lista, o se muestra u oculta una función de interfaz de usuario).

El código HTML de un **formulario web** está compuesto por uno o más **controles de formulario** (a veces llamados **widgets**), además de algunos elementos adicionales que ayudan a estructurar el formulario general (a menudo se los conoce como **formularios HTML**). Los controles pueden ser campos de texto de una o varias líneas, cajas desplegadas, botones, casillas de verificación o botones de opción, y se crean principalmente con el elemento `<input>`, aunque hay algunos otros elementos que también hay que conocer.

Los controles de formulario también se pueden programar para forzar la introducción de formatos o valores específicos (**validación de formulario**), y se combinan con etiquetas de texto que describen su propósito para los usuarios con y sin discapacidad visual.

4. Campos de un formulario

En las próximas secciones crearemos formularios web funcionales. Para ello presentaremos primero algunos controles de formulario y elementos estructurales comunes, y nos centramos también en las mejores prácticas de accesibilidad. A continuación, veremos con detalle las funciones de los diferentes controles de formulario, o *widgets*, y estudiaremos todas las diferentes opciones de que se dispone para la recopilación de diferentes tipos de datos. En esta sección en particular, veremos el conjunto original de controles de formulario, disponible en todos los navegadores desde los primeros días de la web.

4.1. El elemento <label>

El elemento `<label>` nos permite definir una etiqueta para un control de un formulario HTML. Este es el elemento más importante si deseas crear formularios accesibles porque cuando se implementan correctamente, los lectores de pantalla leen la etiqueta de un elemento de formulario junto con las instrucciones relacionadas, y esto además resulta muy útil para los usuarios videntes. Tomemos este ejemplo que vimos en el artículo anterior:

```
1. <label>
2.   Nombre: <input type="text" name="nombre" />
3. </label>
```

Existe otra forma de asociar un control de formulario con una etiqueta. Un elemento `<label>` también se pueden asociar correctamente con un elemento `<input>` por su atributo `for` (que contiene el atributo `id` del elemento `<input>`):

```
1. <label for="nombre">
2.   Nombre: <input type="text" name="nombre" id="nombre" />
3. </label>
```

El resultado en ambos casos será el mismo:

Con la etiqueta `<label>` asociada correctamente con `<input>` por su atributo `for` (que contiene el atributo `id` del elemento `<input>`), un lector de pantalla leerá algo como «Nombre, editar texto». Si no hay ninguna etiqueta, o si el control de formulario no está asociado implícita o explícitamente con alguna etiqueta, un lector de pantalla leerá algo así como «Editar texto en blanco», lo cual no es de mucha ayuda.

4.1.1. ¡También se puede hacer clic en las etiquetas!

Otra ventaja de configurar correctamente las etiquetas es que puedes hacer clic o pulsar en la etiqueta para activar el control de formulario correspondiente. Esto es útil para controles como entradas de texto, donde puedes hacer clic tanto en la etiqueta como en la entrada de texto para proporcionar el foco al control de formulario, pero es útil especialmente para botones de opción y casillas de verificación, porque la zona sensible de este control puede ser muy pequeña, y puede ser útil para facilitar su activación.

Por ejemplo, al hacer clic en el texto de la etiqueta «Me gustan las cerezas» del ejemplo siguiente, cambiará el estado seleccionado de la casilla de verificación *cereza*:

```
1. <label>
2.   <input type="checkbox" name="cereza" value="cereza" />
```



```
3.     Me gustan las cerezas
4.     </label><br />
5.     <label>
6.         <input type="checkbox" name="platano" value="platano" />
7.     Me gustan los plátanos
8.     </label><br />
```

4.2. Campos de entrada de texto

Los campos de texto [<input>](#) son los controles de formulario más básicos, y permiten al usuario introducir cualquier tipo de datos. Pueden tomar muchas formas diferentes según el valor de su atributo [type](#). Se utiliza para crear la mayoría de los tipos de controles de formulario, que incluyen campos de texto de una sola línea, controles para la fecha y la hora, y también controles sin introducción de texto, como casillas de verificación, selectores de opción y selectores de color, e incluso botones.

Todos los controles de texto básicos comparten algunos comportamientos comunes:

- Se pueden marcar como [readonly](#) (el usuario no puede modificar el valor de entrada, pero este se envía con el resto de los datos del formulario) o [disabled](#) (el valor de entrada no se puede modificar y nunca se envía con el resto de los datos del formulario).
- Pueden tener un [placeholder](#) : se trata de un texto que aparece dentro de la caja de entrada de texto y que se usa para describir brevemente el propósito de la caja de texto.
- Pueden presentar una limitación de [tamaño](#) (el tamaño físico de la caja de texto) y de la [extensión máxima](#) (el número máximo de caracteres que se pueden poner en la caja de texto).
- Permiten [corrección ortográfica](#) (utilizando el atributo [spellcheck](#)), si el navegador la admite.

Debemos tener en cuenta que los campos de texto de los formularios HTML son controles de entrada de texto sencillos sin formato. Esto significa que no puedes usarlos para aplicar una [edición enriquecida](#) (negrita, cursiva, etc.). Todos los controles de formulario que encuentres con texto enriquecido son controles de formulario personalizados creados con HTML, CSS y JavaScript.

4.2.1. Campos de texto de una sola línea

Un campo de texto de una sola línea se crea utilizando un elemento [<input>](#) cuyo valor de atributo [type](#) se establece en `text` , u omitiendo por completo el atributo [type](#) (`text` es el valor predeterminado). El valor `text` de este atributo también es el valor alternativo si el navegador no reconoce el valor que has especificado para el atributo [type](#) (por ejemplo, si

especificas `type="color"` y el navegador no está dotado en origen de un control de selección de colores).

Veamos este ejemplo en el que aparecen un par de campos de texto de una sola línea:

```
1.  <label>
2.    Nombre (de 5 a 10 caracteres):
3.    <input type="text" name="nombre" required
4.          minlength="5" maxlength="10" size="15"
5.          placeholder="Escriba aquí su nombre">
6.  </label><br />
7.  <label>
8.    Comentario:
9.    <input type="text" name="comentario" required
10.         placeholder="Escriba aquí su comentario">
11.  </label><br />
```

HTML5 ha mejorado el campo de texto básico original de una sola línea al añadir valores especiales para el atributo `type` que imponen restricciones de validación específicas y otras características, por ejemplo, específicas para introducir direcciones URL o números. Los expondremos más adelante, y también puedes encontrar más información en el siguiente enlace: [Los tipos de entrada en HTML5](#).

4.2.1.1. Campo de contraseña

Uno de los tipos de entrada originales era el tipo de campo de texto `password` :

```
1.  <label>
2.    Contraseña: <input type="password" name="contrasena">
3.  </label>
```

El valor de la contraseña no añade restricciones especiales al texto que se introduce, pero oculta el valor que se introduce en el campo (por ejemplo, con puntos o asteriscos) para impedir que otros puedan leerlo.

Ten en cuenta que esto sólo se aplica a nivel de interfaz de usuario; a menos que envíes tu formulario en modo seguro, se enviará como texto plano, lo que es malo desde el punto de vista de la seguridad porque alguien con malas intenciones podría interceptar tus datos y robar tus contraseñas, datos de tarjetas de crédito o cualquier otra cosa que hayas enviado. La mejor manera de proteger a los usuarios de esto es alojar cualquier página que contenga formularios en una ubicación de conexión segura (es decir, en una dirección `https://`), de modo que los datos se cifren antes de enviarse.

Los navegadores reconocen las implicaciones de seguridad de enviar datos de formulario por una conexión insegura y disponen de mensajes de advertencia para disuadir a los usuarios de usar formularios no seguros. Para obtener más información sobre las implementaciones de Firefox al respecto, consulta el artículo [Contraseñas inseguras](#).

4.2.2. Campos de texto de varias líneas

El elemento `<textarea>` representa un control de edición de texto de varias líneas, útil cuando deseas permitir que los usuarios introduzcan una cantidad considerable de texto, por ejemplo, un comentario de una revisión o en un formulario de contactar:

```
1. <label>Cuéntanos tu historia:
2.   <textarea name="historia" rows="5">
3.     Era una noche oscura y tormentosa...
4.   </textarea>
5. </label>
```

Puede usar los atributos `rows` y `cols` para especificar el tamaño exacto para el campo `<textarea>`. Establecer estos a veces es una buena idea para mantener la coherencia, ya que los valores predeterminados del navegador pueden diferir. También estamos usando un contenido predeterminado (el que aparece entre las etiquetas de apertura y cierre), ya que `<textarea>` no admite el atributo `value`, tal como veremos en la próxima sección.

El elemento `<textarea>` también acepta algunos atributos comunes de los que también disponen los elementos `<input>`, tales como `autocomplete`, `autofocus`, `disabled`, `placeholder`, `readonly`, and `required`.

4.2.3. Valores por defecto en los cuadros de texto

A continuación vamos a exponer una de las rarezas que presenta HTML respecto a la sintaxis de `<input>` en contraposición con la de `<textarea>`. La etiqueta `<input>` es un elemento vacío, lo que significa que no necesita una etiqueta de cierre. Sin embargo, el elemento `<textarea>` no es un elemento vacío, lo que significa que debe cerrarse con la etiqueta de cierre adecuada. Esto tiene un impacto en una característica específica de los formularios: el modo en que se define el valor predeterminado. Para definir el valor por defecto de un elemento `<input>`, debemos usar el atributo `value` de esta manera:

```
1. <input type="text" value="por defecto este elemento se llena con este texto">
```

Por otro lado, si queremos definir un valor predeterminado para un elemento `<textarea>`, debemos colocarlo entre las etiquetas de apertura y cierre del elemento `<textarea>`, de esta

forma:

1. `<textarea>`
2. Por defecto, este elemento contiene este texto
3. `</textarea>`

4.3. Casillas de verificación y botones de opción

Los elementos de selección (o *checkable items*, en inglés) son controles cuyo estado puede cambiar cuando se hace clic en ellos o en sus etiquetas asociadas. Hay dos tipos de elementos de selección: las casillas de verificación (o *check boxes*) y los botones de opción (o *radio buttons*). Ambos usan el atributo `checked` para indicar si el control de formulario está seleccionado por defecto o no.

Vale la pena señalar que estos controles no se comportan exactamente como otros controles de formulario. Para la mayoría de los controles de formulario, cuando se envía el formulario, se envían todos los controles que tienen un atributo `name`, incluso si en ellos no se ha introducido ningún valor. En el caso de elementos de selección, sus valores se envían solo si están seleccionados. Si no están seleccionados, no se envía nada, ni siquiera su nombre. Si están seleccionados pero no tienen ningún valor asignado, el nombre se envía con el valor *on*.

4.3.1. Casillas de verificación

Las casillas de verificación se crean estableciendo el atributo `type` del elemento `<input>` con el valor `checkbox`. Los elementos de este tipo se suelen representar como casillas que se marcan al activarse, como las que se pueden ver en muchos formularios oficiales en papel. La apariencia exacta depende de la configuración del sistema operativo en el que se ejecuta el navegador. Generalmente se trata de un cuadrado, pero puede tener esquinas redondeadas. Una casilla de verificación permite seleccionar diversas opciones para enviarlas en un formulario.

Veamos y probemos un ejemplo muy simple:

1. `<label>`
2. `<input type="checkbox" name="zanahorias" value="zanahorias" checked />`
3. `¿Te gustan las zanahorias?`
4. `</label>`

Al incluir el atributo `checked`, la casilla de verificación se marca automáticamente cuando se carga la página. Al hacer clic en la casilla de verificación o en su etiqueta asociada, la

casilla de verificación se activa o desactiva.

Debido a su naturaleza activa-inactiva, las casillas de verificación se consideran botones de conmutación, y muchos desarrolladores y diseñadores han ampliado el estilo de casilla de verificación predeterminado para crear botones que parecen interruptores de conmutación. Aquí puedes ver un [ejemplo](#) (y también puedes observar el [código fuente](#)).

4.3.2. Botones de opción

Un botón de opción se crea estableciendo el atributo `type` del elemento `<input>` en el valor `radio`. Los elementos de este tipo se utilizan generalmente en grupos (colecciones de botones del mismo tipo que describen un conjunto de opciones relacionadas). Solo se puede seleccionar un botón de opción en un grupo determinado al mismo tiempo. Los botones de opción se representan normalmente como círculos pequeños, que se rellenan o resaltan cuando se seleccionan.

Veamos el código fuente de un ejemplo simple que contiene varios botones de opción y cómo lo representa un navegador:

```
1. ¿Cuál es tu comida favorita?<br />
2. <label>
3.   <input type="radio" name="comida" value="sopa" checked="" />Sopa
4. </label><br />
5. <label>
6.   <input type="radio" name="comida" value="curry" />Curry
7. </label><br />
8. <label>
9.   <input type="radio" name="comida" value="pizza" />Pizza
10. </label><br />
```

Como hemos visto en este último ejemplo, es posible asociar diversos botones de opción. Si comparten el mismo valor de atributo `name`, se considera que están en el mismo grupo de botones. Solo un botón dentro de un grupo puede estar activado en cada momento. Esto significa que cuando uno de ellos se selecciona, todos los demás se deselectan automáticamente. Al enviar el formulario, solo se envía el valor del botón de opción seleccionado. Si ninguno de ellos está seleccionado, se considera que el conjunto completo de botones de opción está en un estado desconocido y no se envía ningún valor con el formulario. Cuando en un grupo de botones con el mismo nombre se selecciona uno de los botones de opción, no es posible deselectar todos los botones sin reiniciar el formulario.

4.4. El elemento <select>

El elemento `select` (`<select>`) de HTML representa un control que muestra un menú de opciones. Las opciones contenidas en el menú son representadas por elementos `<option>` , los cuales pueden ser agrupados por elementos `<optgroup>` . Además, una determinada opción puede estar preseleccionada al cargarse la página.

Veamos el siguiente ejemplo de código y cómo lo representa el navegador:

```
1.  <label>Escoge la mascota que más te guste:
2.    <select name="mascota" id="mascota">
3.      <option value="">--Selecciona una opción--</option>
4.      <option value="perro">Perro</option>
5.      <option value="gato">Gato</option>
6.      <option value="hamster">Hamster</option>
7.      <option value="loro">Loro</option>
8.      <option value="arana">Araña</option>
9.      <option value="pez">Pez</option>
10.    </select>
11.  </label>
```

El ejemplo anterior muestra el uso típico del elemento `<select>` . Lo asociamos con una etiqueta `<label>` con fines de accesibilidad y utilizamos el atributo `name` para representar el nombre del campo que se enviará al servidor. Cada opción de menú está definida por un elemento `<option>` anidado dentro del `<select>` .

Cada elemento `<option>` debe tener un atributo `value` que contenga el valor para enviar al servidor cuando se seleccione esa opción. Si no se incluye ningún atributo `value` , el valor predeterminado es el texto contenido dentro del elemento. Puedes incluir un atributo `selected` en un elemento `<option>` para que se seleccione de forma predeterminada cuando se carga la página por primera vez.

El elemento `<select>` tiene algunos atributos únicos que puedes usar para personalizarlo. Por ejemplo, con el atributo `multiple` especificas si se pueden seleccionar varias opciones y con el atributo `size` puedes especificar cuántas opciones deben mostrarse a la vez. También se aceptan muchos atributos genéricos tales como `required` , `disabled` , `autofocus` , etc.

Por último, conviene mencionar que se pueden anidar varios elementos `<option>` dentro de `<optgroup>` para crear grupos separados de opciones dentro del menú desplegable.

4.5. Selector de archivos

Hay un último tipo de `<input>` : el tipo entrada de archivo. Los formularios pueden enviar archivos a un servidor (esta acción específica también se detalla en el artículo [Enviar los datos del formulario](#)). El control de selección de archivos se puede usar para elegir uno o más archivos para enviar.

Para crear un [control de selección de archivos](#), podemos utilizar el elemento `<input>` con su atributo `type` establecido en `file`. Es posible restringir los tipos de archivos que se aceptan utilizando el atributo `accept`. Además, puedes permitir que el usuario elija más de un archivo añadiendo el atributo `multiple`.

En este ejemplo, se crea un control de selección de archivos que solicita archivos de imágenes gráficas. En este caso, el usuario puede seleccionar múltiples archivos:

```
1. <input type="file" name="file" accept="image/*" multiple="" />
```

En algunos dispositivos móviles, el control de selección de archivos permite acceder a fotos, vídeos y audio capturados directamente por la cámara y el micrófono del dispositivo. Para ello basta con añadir información de captura al atributo `accept` de la manera siguiente:

```
1. <input type="file" accept="image/*;capture=camera">
2. <input type="file" accept="video/*;capture=camcorder">
3. <input type="file" accept="audio/*;capture=microphone">
```

4.6. Botones

El botón de opción no es realmente un botón, a pesar de su nombre. A continuación echaremos un vistazo a los controles de formulario que son botones propiamente. La etiqueta de HTML `<button>` representa un elemento de tipo botón que puede ser utilizado en formularios o en cualquier parte de la página que necesite un botón simple y estándar para iniciar una acción al pulsar sobre él. De forma predeterminada, los botones HTML se presentan con un estilo similar en todas las plataformas, pero estos estilos se pueden cambiar fácilmente utilizando [CSS](#).

El comportamiento por defecto de un botón se puede cambiar mediante el atributo `type`. Los posibles valores que podemos utilizar son:

- `submit` : El botón envía los datos del formulario al servidor. Este es el valor predeterminado si el atributo no se especifica para los botones asociados con el formulario o si el atributo contiene un valor vacío o no válido.

- `reset` : El botón restablece todos los controles a sus valores iniciales. Debería usarse solo cuando sea necesario, ya que puede provocar que un usuario pierda los datos que acaba de introducir.
- `button` : El botón no tiene un comportamiento predeterminado y no hace nada cuando se presiona de manera predeterminada. Utilizando código JavaScript podemos iniciar diversas acciones para responder a los eventos que genere este elemento.

Veamos algunos tipos de botones con un ejemplo sencillo:

```

1.  <p>
2.    <label>Introduce un comentario: <input type="text" name="comentario"
    required="" /></label>
3.  </p>
4.  <p>
5.    <button type="submit">Este es un botón de tipo "submit"</button>
6.  </p>
7.  <p>
8.    <button type="reset">Este es un botón de tipo "reset"</button>
9.  </p>
10. <p>
11.   <button type="button">Este es un botón de tipo "button"</button>
12. </p>

```

Como puedes ver en los ejemplos, los elementos `<button>` te permiten usar código HTML entre las etiquetas `<button>` de apertura y cierre. Los elementos `<input>`, por otro lado, son elementos vacíos; el contenido que muestran está limitado al atributo `value` y, por lo tanto, solo aceptan contenido de texto sin formato.

4.7. Ejercicio propuesto: Campos básicos

Crea una página web para mostrar ejemplos de los elementos de tipo «input» de esta unidad: texto de una sola línea y de varias líneas, contraseña, casillas de verificación y botones de radio, lista desplegable con opciones y selector de archivos. Debes incluir al menos un ejemplo de cada uno de ellos. Debes usar párrafos y etiquetas, y también el atributo «required» y otras posibles restricciones que se puedan aplicar a cada uno de ellos. Verifica el resultado en tu navegador y no olvides incluir todas las etiquetas HTML básicas y validar tu código. Por último, verifica el resultado en tu teléfono móvil.

— Nota: Coloca todas las etiquetas dentro de un contenedor `<form>` y usa un botón de tipo «submit» para verificar que los campos estén validados correctamente. Puedes utilizar un código similar a este:

```
1. <form>
2.   <p><label>
3.     Nombre: <input type="text" name="nombre" required="" />
4.   </label></p>
5.   <p><label>
6.     Apellidos: <input type="text" name="apellidos" required="" />
7.   </label></p>
8.   <p><label>
9.     Contraseña: <input type="password" name="contrasena1" required="" />
10.  </label></p>
11.  <p><label>
12.    Repite la contraseña: <input type="password" name="contrasena2"
13.      required="" />
14.  </label></p>
15.  ...
16.  <p><button>Enviar</button></p>
17. </form>
```

5. Campos nuevos en HTML5

En la sección anterior vimos el elemento `<input>` y los valores de su atributo `type`, disponibles desde los inicios de HTML. Ahora veremos en detalle la funcionalidad de los controles de formulario más recientes, incluyendo algunos tipos de entrada nuevos, los cuales fueron añadidos en HTML5 para permitir la recolección de tipos de datos específicos.

5.1. Campo de dirección de correo electrónico

Este tipo de campo se define utilizando el valor `email` en el atributo `type` del elemento `<input>`:

```
1. <label>
2.   Introduce un correo electrónico válido:
3.   <input type="email" name="correo" placeholder="prueba@prueba.com"
4.     required="" />
5. </label>
```

Cuando se utiliza este valor `type`, se le obliga al usuario a escribir dentro del campo una dirección de correo electrónico válida. Cualquier otro contenido ocasiona que el navegador muestre un mensaje de error cuando se envía el formulario. Puedes verlo en acción en el siguiente ejemplo:

Puedes utilizar también el atributo `multiple` en combinación con el tipo `input email` para permitir que sean introducidas varias direcciones de correo electrónico separadas por comas en el mismo input:

1. `<label>`
2. Múltiples correos electrónicos: `<input type="email" name="correos" multiple="" />`
3. `</label>`

En algunos dispositivos, en particular dispositivos táctiles con teclados dinámicos como los smart phones, debería desplegarse un teclado virtual que es más adecuado para introducir direcciones de correo electrónico, incluyendo la tecla @.

5.2. Campo URL

Se puede crear un tipo especial de campo para introducir URLs utilizando el valor `url` para el atributo `type`:

1. `<label>`
2. Introduce una URL:
3. `<input type="url" name="url" placeholder="https://..." required="" />`
4. `</label>`

Este tipo añade restricciones de validación en el campo. El navegador informará de un error si no se introdujo el protocolo (como `http:` o `https:`), o si de algún modo la URL es claramente incorrecta. Debemos tener en cuenta que solo porque la URL tenga un formato correcto, no significa necesariamente que la dirección al que hace referencia exista. Puedes hacer pruebas con el siguiente ejemplo:

En dispositivos con teclados dinámicos a menudo se mostrarán por defecto algunas o todas las teclas como los dos puntos, el punto, y la barra inclinada.

5.3. Campo número de teléfono

Se puede crear un campo especial para introducir números de teléfono utilizando `tel` como valor del atributo `type`:

1. `<label>`
2. Introduce un número de teléfono:
3. `<input type="tel" name="telefono" placeholder="123 456 789" />`
4. `</label>`

Cuando se accede desde un dispositivo táctil con teclados dinámicos, muchos de ellos mostrarán un teclado numérico cuando se encuentren con `type="tel"`, lo que significa que este tipo es útil no sólo para ser utilizado para números de teléfono, sino también cuando sea útil un teclado numérico.

Debido a la gran variedad de formatos de número de teléfono existentes, este tipo de campo no establece ningún tipo de restricción sobre el valor introducido por el usuario (esto significa que permite incluir letras, etc...). Se puede utilizar el atributo [pattern](#) para establecer restricciones (puedes consultar más información sobre este atributo en [Validación de formulario en el lado cliente](#)).

5.4. Campo numérico

Se pueden crear controles para introducir números con el `type` `number` de `<input>`. Este control se parece a un campo de texto pero solo permite números de punto flotante, y normalmente proporciona botones deslizadores para incrementar o reducir el valor del control. En dispositivos con teclados dinámicos generalmente se muestra el teclado numérico.

Con el tipo de `input` `number` puedes limitar los valores mínimo y máximo permitidos definiendo los atributos `min` y `max`. También puedes utilizar el atributo `step` para cambiar el incremento y decremento causado por los botones deslizadores. Por defecto, el tipo de `input` `number` sólo valida si el número es un entero. Para permitir números decimales, especifica `step="any"`. Si se omite, su valor por defecto es `1`, lo que significa que solo son válidos números enteros.

Miremos algunos ejemplos. El primero de los siguientes crea un control numérico cuyo valor está restringido a cualquier valor entre `1` y `10`, y sus botones cambian su valor en incrementos o decrementos de `2`.

```
1. <input type="number" name="edad" min="1" max="10" step="2" value="1" />
```

El segundo crea un control numérico cuyo valor está restringido a cualquier valor entre el `0` y `1` ambos inclusive, y sus botones cambian su valor en incrementos o decrementos de `0.01`:

```
1. <input type="number" name="cambio" min="0" max="1" step="0.01" value="0" />
```

El tipo de `input` `number` tiene sentido cuando esté limitado el rango de valores válidos, por ejemplo la edad de una persona o su altura. Si el rango es demasiado grande para que los cambios de incremento tengan sentido (por ejemplo los códigos postales de USA, cuyo rango va de `00001` a `99999`), entonces sería una mejor opción utilizar el tipo `tel`: proporciona el teclado numérico mientras que omite el componente de interfaz de usuario de los deslizadores de número.

5.5. Controles deslizantes

Otra forma de introducir un número es usando un **slider**. Habrás observado bastantes controles parecidos en webs donde por ejemplo se tiene que establecer un precio máximo para realizar una compra y se solicita realizar un filtro por dicho campo. Vamos a observar un ejemplo:

En cuanto al uso, los controles deslizantes son menos precisos que los campos de texto. Por lo tanto, se utilizan para elegir un número cuyo valor *exacto* no es necesariamente importante.

Un control deslizante o barra de desplazamiento se crea usando el `<input>` con el valor `range` en el atributo `type`. El deslizador se puede mover con el ratón, también utilizando dispositivos táctiles, o con las flechas del teclado.

Es importante configurar correctamente este campo, y con ese propósito es muy recomendable utilizar los atributos `min`, `max` y `step`, que establecen los valores mínimo, máximo y de incremento, respectivamente.

Veamos el código fuente correspondiente al ejemplo anterior, para que puedas ver cómo se consigue dicho resultado. En primer lugar, el código HTML básico:

```
1. <form>
2.   <p>Escoge un precio máximo:</p>
3.   <input type="range" name="barra"
4.         min="50000" max="500000" step="100" value="250000"
5.         oninput="numero.value = this.value" />
6.   <input type="number" name="numero"
7.         min="50000" max="500000" step="100" value="250000"
8.         oninput="barra.value = this.value" />
9. </form>
```

Este ejemplo crea un control deslizante cuyo valor puede oscilar entre 50000 y 500000, que aumenta / disminuye de 100 en 100. En este caso hemos asignado el valor predeterminado de 250000, usando el atributo `value`.

Un inconveniente que presentan los controles deslizantes es que no ofrecen ningún tipo de retroalimentación visual sobre cuál es el valor actual. Por eso hemos incluido un elemento `<input>` para mostrar (o también cambiar) el valor actual.

5.6. Selectores de fecha y hora

La recopilación de valores de fecha y hora ha sido desde siempre una pesadilla para los desarrolladores web. Para una buena experiencia de usuario, es importante proporcionar una interfaz que permita seleccionar directamente una fecha sobre un calendario, sin necesidad de cambiar de contexto a una aplicación de calendario nativa o a otra aplicación que utilice diferentes formatos que sean difíciles de analizar. Por ejemplo, el último minuto del milenio anterior se puede expresar de las siguientes formas diferentes, por ejemplo: 1999/12/31, 23:59 o 12/31/99T11:59PM.

Los controles de fecha HTML están disponibles para manejar este tipo específico de datos, proporcionando widgets de calendario y haciendo que los datos sean uniformes.

Para crear un campo de tipo fecha utilizamos el elemento `<input>` y un valor apropiado para el atributo `type`, dependiendo de si deseas recopilar fechas, horas o ambos. Aquí puedes observar un ejemplo (si el navegador no aceptara alguno de estos tipos, utilizaría un elemento alternativo que permita la introducción mediante texto):

```
1. <p><label>
2.   Fecha y hora local: <input type="datetime-local" name="fechayhora" />
3. </label></p>
4. <p><label>
5.   Mes: <input type="month" name="mes">
6. </label></p>
7. <p><label>
8.   Hora: <input type="time" name="hora">
9. </label></p>
10. <p><label>
11.   Semana: <input type="week" name="semana">
12. </label></p>
```

Todos los controles de fecha y hora se pueden restringir usando los atributos `min` y `max`, con una mayor restricción posible a través del atributo `step` (cuyo valor varía según el tipo de entrada):

```
1. <label>
2.   ¿Cuándo es tu cumpleaños?
3.   <input type="date" name="fecha" min="1975-01-01" max="2025-12-31" step="1"
4.   />
5. </label>
```

5.7. Selector de color

Los colores también suelen ser difíciles de manejar. Hay muchas formas de expresarlos: valores RGB (decimal o hexadecimal), valores HSL, palabras clave, etc.

Se puede crear un control de tipo `color` utilizando el elemento `<input>` con su atributo `type` con valor `color` :

1. `<label>`
2. Selecciona una color: `<input type="color" name="color" />`
3. `</label>`

Si el navegador admite este tipo de campo, al hacer clic en él se utilizará la funcionalidad de elección de color predeterminada del sistema operativo. Puedes observar el resultado con el siguiente ejemplo:

5.8. Campo de búsqueda

Los campos de búsqueda se utilizan para crear cajas de búsqueda en páginas web y aplicaciones. Este tipo de campo se define utilizando el valor `search` en su atributo `type` :

1. `<input type="search" name="buscar" placeholder="Buscar" required="" />`

La diferencia principal entre un campo `text` y un campo `search` , es el estilo que el navegador aplica a uno u otro. A menudo los campos `search` se muestran con bordes redondeados; y a veces también se muestra una «ⓧ», para limpiar el campo de cualquier valor cuando se pulsa sobre él. Además, en dispositivos con teclado dinámico, la tecla enter del teclado suele mostrar un icono de lupa.

Otra característica que vale la pena señalar es que se pueden guardar los valores de un campo `search` automáticamente y reutilizarse en múltiples páginas del mismo sitio web para ofrecer autocompletado. Esta característica suele ocurrir de forma automática en la mayoría de navegadores modernos.

5.9. Ejercicio propuesto: Campos HTML5

Crea una página web para mostrar todos los elementos de esta última sección: correo electrónico, url, número de teléfono, campo numérico, control deslizante, fecha y hora, selector de color y campo de búsqueda. Debes incluir al menos un ejemplo para cada uno de ellos. Debes usar párrafos y etiquetas, y también el atributo «required» y otras restricciones

según el campo del que se trate. Comprueba el resultado en tu navegador y no olvides incluir todas las etiquetas HTML básicas y validar el código. Por último, comprueba el resultado también en tu teléfono móvil.

— Coloca todas las etiquetas dentro de un contenedor `<form>` y usa un botón de tipo «submit» para comprobar que los campos se validan correctamente:

```
1. <form>
2.   <p><label>
3.     Correo principal: <input type="email" name="correo1" required="" />
4.   </label></p>
5.   <p><label>
6.     Otro correo: <input type="email" name="correo2" required="" />
7.   </label></p>
8.   <p><label>
9.     Tu página web: <input type="url" name="web1" required="" />
10.  </label></p>
11.  <p><label>
12.    Página web de tu instituto: <input type="url" name="web2" required="" />
13.  </label></p>
14.  ...
15.  <p><button>Enviar</button></p>
16. </form>
```

6. Atributos comunes

Muchos de los elementos que se utilizan para definir controles de formulario tienen sus atributos específicos propios. Sin embargo, hay un conjunto de atributos que son comunes para todos los elementos de formulario. Ya has conocido algunos, pero a continuación encontrarás una lista de esos atributos comunes para referencias futuras:

Nombre del atributo	Valor por defecto	Descripción
autofocus	false	Este atributo booleano te permite especificar que el elemento ha de tener el foco de entrada automáticamente cuando se carga la página. En un documento, solo un elemento asociado a un formulario puede tener este atributo especificado.
disabled	false	Este atributo booleano indica que el usuario no puede interactuar con el elemento. Si no se especifica este atributo, el elemento hereda su configuración del elemento que lo contiene, por ejemplo, <code><fieldset></code> . Si el elemento que lo contiene no tiene el atributo establecido en <code>disabled</code> , el elemento está habilitado.

Nombre del atributo	Valor por defecto	Descripción
form		El elemento <code><form></code> con el que está asociado el control, que se usa cuando no está anidado dentro de ese formulario. El valor del atributo debe ser el atributo <code>id</code> de un elemento <code><form></code> del mismo documento. Esto te permite asociar un formulario con un control de formulario que esté fuera de aquel, incluso si está dentro de un elemento de formulario diferente.
name		El nombre del campo que se envía con los datos del formulario.
value		El valor inicial del campo.

7. Un primer formulario «real»

En esta sección crearemos un formulario web completo y funcional. Diseñaremos un formulario sencillo con los campos adecuados y otros elementos HTML, y explicaremos cómo se envían los datos a un servidor. Ampliaremos cada uno de estos subtemas más adelante.

7.1. Diseñar el formulario

Antes de comenzar a escribir código, siempre es mejor tomarse el tiempo necesario para pensar cómo queremos que sea nuestro formulario. Diseñar una maqueta rápida nos ayudará a definir el conjunto de datos adecuado que debemos pedirle al usuario. Desde el punto de vista de la experiencia del usuario (UX), es importante recordar que cuanto más grande es un formulario, más te arriesgas a frustrar a las personas que lo usen. Un

formulario tiene que ser simple y conciso, y por ello deberíamos solicitar solo los datos que necesitemos.

Diseñar formularios es un paso importante cuando creamos un sitio web o una aplicación. Va más allá del alcance de esta sección exponer esto en detalle, pero si deseas profundizar en ese tema, puedes leer los siguientes artículos

- Smashing Magazine tiene algunos [artículos muy buenos sobre formularios UX](#), incluido el artículo —antiguo pero relevante— [An Extensive Guide To Web Form Usability](#) [Amplia guía de usabilidad para formularios web].
- UXMatters también es un recurso que da buenos consejos, desde [buenas prácticas básicas](#) hasta cuestiones complejas como los [formularios de varias páginas](#).

En esta sección vamos a crear un formulario de contacto sencillo. Observemos un posible borrador:

Nuestro formulario va a tener tres campos de texto y un botón. Le pedimos al usuario su nombre, su correo electrónico y el mensaje que desea enviar. Al pulsar el botón sus datos se enviarán a un servidor web.

7.2. Implementando nuestro formulario

Vamos a proceder ahora a escribir el código HTML para nuestro formulario. Vamos a utilizar los elementos HTML siguientes: `<form>` , `<label>` , `<input>` , `<textarea>` y `<button>` .

7.3. El elemento `<form>`

Todos los formularios comienzan con un elemento `<form>` , como este:

1. `<form action="contactar.php" method="get">`
- 2.
3. `</form>`

Este elemento define formalmente un formulario. Es un elemento contenedor, como un elemento `<section>` o `<footer>` , pero específico para contener formularios; también admite algunos atributos específicos para la configuración de la forma en que se comporta el formulario. Todos sus atributos son opcionales, pero es una práctica estándar establecer siempre al menos los atributos `action` y `method` :

- El atributo `action` define la ubicación (URL) donde se envían los datos que el formulario ha recopilado cuando se validan.
- El atributo `method` define con qué método HTTP se envían los datos (generalmente `get` o `post`).

Veremos cómo funcionan esos atributos más adelante.

7.4. Los elementos `<label>`, `<input>` y `<textarea>`

Nuestro formulario de contacto no es complejo en absoluto: la parte para la entrada de datos contiene tres campos de texto, cada uno con su elemento `<label>` correspondiente:

- El campo de entrada para el nombre es un [campo de texto de una sola línea](#).
- El campo de entrada para el correo electrónico es una [entrada de datos de tipo correo electrónico](#): un campo de texto de una sola línea que acepta solo direcciones de correo electrónico.
- El campo de entrada para el mensaje es `<textarea>`; un campo de texto multilínea.

En términos de código HTML, para implementar estos controles de formulario necesitamos algo como lo siguiente:

```

1.  <form action="contactar.php" method="GET">
2.    <p>
3.      <label>Nombre: <input type="text" name="nombre" required="" /></label>
4.    </p>
5.    <p>
6.      <label>Correo: <input type="email" name="correo" required="" /></label>
7.    </p>
8.    <p>
9.      <label>Mensaje: <textarea name="mensaje" required=""></textarea></label>
10.   </p>
11. </form>

```

Por motivos de usabilidad y accesibilidad incluimos una etiqueta explícita para cada control de formulario. Hacer esto presenta muchas ventajas porque la etiqueta está asociada al control del formulario y permite que los usuarios con ratón, y dispositivos táctiles hagan clic en la etiqueta para activar el control correspondiente, y también proporciona accesibilidad con un nombre que los lectores de pantalla leen a sus usuarios. Puedes encontrar más detalles sobre las etiquetas de los formularios en [Cómo estructurar un formulario web](#).

En el elemento `<input>`, el atributo más importante es `type`, ya que define la forma en que el elemento `<input>` aparece y se comporta. Puedes encontrar más información sobre esto

en el artículo sobre [Controles de formularios nativos básicos](#).

- En nuestro ejemplo, usamos el valor `<input/text>` para la primera entrada, el valor predeterminado para este atributo. Representa un campo de texto básico de una sola línea que acepta cualquier tipo de entrada de texto.
- Para la segunda entrada, usamos el valor `<input/email>`, que define un campo de texto de una sola línea que solo acepta una dirección de correo electrónico. Esto convierte un campo de texto básico en una especie de campo «inteligente» que efectúa algunas comprobaciones de validación de los datos que el usuario escribe. También hace que aparezca un diseño de teclado más apropiado para introducir direcciones de correo electrónico (por ejemplo, con un símbolo @ por defecto) en dispositivos con teclados dinámicos, como teléfonos inteligentes. Puedes encontrar más información sobre la validación de formularios en el artículo de [Validación de formularios por parte del cliente](#).

7.5. El elemento `<button>`

El marcado de nuestro formulario está casi completo; solo necesitamos añadir un botón para permitir que el usuario envíe sus datos una vez que haya completado el formulario. Esto se hace con el elemento `<button>`; basta con añadir lo siguiente justo encima de la etiqueta de cierre `</form>`:

```
1. <button type="submit">Enviar mensaje</button>
```

Como hemos explicado anteriormente, el elemento `<button>` también acepta el atributo `type`, que a su vez acepta uno de estos tres valores: `submit`, `reset` o `button`:

- Un clic en un botón `submit` (el valor predeterminado) envía los datos del formulario a la página web definida por el atributo `action` del elemento `<form>`.
- Un clic en un botón `reset` restablece de inmediato todos los controles de formulario a su valor predeterminado. Desde el punto de vista de UX, esto se considera una mala práctica, por lo que debes evitar usar este tipo de botones a menos que realmente tengas una buena razón para incluirlos.
- Un clic en un botón `button` no hace... ¡nada! Eso suena tonto, pero es muy útil para crear botones personalizados: puedes definir su función con JavaScript.

7.6. Enviando datos al servidor web

La última parte, y quizás la más complicada, es manejar los datos del formulario en el lado del servidor. El elemento `<form>` define dónde y cómo enviar los datos gracias a los atributos `action` y `method`.

Proporcionamos un nombre (`name`) a cada control de formulario. Los nombres son importantes tanto en el lado del cliente como del servidor; le dicen al navegador qué nombre debe dar a cada dato y, en el lado del servidor, permiten que el servidor acceda a cada dato utilizando su nombre. Los datos del formulario se envían al servidor como pares de nombre/valor.

Para poner nombre a los diversos datos que se introducen en un formulario, debes usar el atributo `name` en cada control de formulario que recopila un dato específico. Veamos de nuevo algunos el código fuente de nuestro formulario:

```
1.  <form action="contactar.php" method="GET">
2.    <p>
3.      <label>Nombre: <input type="text" name="nombre" required="" /></label>
4.    </p>
5.    <p>
6.      <label>Correo: <input type="email" name="correo" required="" /></label>
7.    </p>
8.    <p>
9.      <label>Mensaje: <textarea name="mensaje" required=""></textarea></label>
10.    </p>
11.    <p>
12.      <button type="submit">Enviar mensaje</button>
13.    </p>
14.  </form>
```

En nuestro ejemplo, el formulario envía tres datos denominados « nombre », « correo » y « mensaje ». Esos datos se envían a la URL « `contactar.php` » utilizando el método [GET de HTTP](#).

En el lado del servidor, la secuencia de comandos de la URL « `contactar.php` » recibe los datos como una lista de tres elementos clave/valor contenidos en la solicitud HTTP. La forma en que este script maneja esos datos depende de ti. Cada lenguaje de servidor (PHP, Python, Ruby, Java, C#, etc.) tiene su propio mecanismo de manipulación de datos de formulario. No profundizaremos demasiado en esta sección, pero intentaremos completar nuestro ejercicio para que sea completamente funcional. También puedes consultar el artículo [Enviar los datos de un formulario](#).

7.6.1. Validación de formulario en la parte del cliente

Antes de enviar datos al servidor, es importante asegurarse de que se completan todos los controles de formulario requeridos, y en el formato correcto. Esto se denomina **validación de formulario en el lado del cliente** y ayuda a garantizar que los datos que se envían coinciden con los requisitos establecidos en los diversos controles de formulario.

La validación en el lado del cliente es una verificación inicial y una característica importante para garantizar una buena experiencia de usuario; mediante la detección de datos no válidos en el lado del cliente, el usuario puede corregirlos de inmediato. Si el servidor lo recibe y, a continuación, lo rechaza; se produce un retraso considerable en la comunicación entre el servidor y el cliente que insta al usuario a corregir sus datos.

Si accedes a cualquier sitio web que incluya un formulario de registro observarás que se muestran comentarios cuando no introduces tus datos en el formato que se espera. Recibirás mensajes como:

- «Este campo es obligatorio» (No se puede dejar este campo en blanco).
- «Introduzca su número de teléfono en el formato xxx-xxx-xxx» (Se requiere un formato de datos específico para que se considere válido).
- «Introduzca una dirección de correo electrónico válida» (los datos que introdujiste no están en el formato correcto).
- «Su contraseña debe tener entre 8 y 30 caracteres y contener una letra mayúscula, un símbolo y un número». (Se requiere un formato de datos muy específico para tus datos).

Esto se llama **validación de formulario**. Cuando introduces los datos, el navegador y/o el servidor web verifican que estén en el formato correcto y dentro de las restricciones establecidas por la aplicación. La validación realizada en el navegador se denomina **validación en el lado del cliente**, mientras que la validación realizada en el servidor se denomina **validación en el lado del servidor**. En esta sección nos centraremos en la validación en el lado del cliente.

Si la información está en el formato correcto, la aplicación permite que los datos se envíen al servidor y (en general) que se guarden en una base de datos; si la información no está en el formato correcto, da al usuario un mensaje de error que explica lo que debe corregir y le permite volver a intentarlo.

Una de las características más importantes de los [controles de formulario de HTML5](#) es la capacidad de validar la mayoría de los datos de usuario sin depender de JavaScript. Esto se realiza mediante el uso de atributos de validación en los elementos del formulario. Los hemos visto anteriormente, pero recapitulamos aquí:

- [required](#) : Especifica si un campo de formulario debe completarse antes de que se pueda enviar el formulario.
- [minlength](#) y [maxlength](#) : Especifican la longitud mínima y máxima de los datos de texto (cadenas).
- [min](#) y [max](#) : Especifican los valores mínimo y máximo de los tipos de entrada numéricos.
- [type](#) : Especifica si los datos deben ser un número, una dirección de correo electrónico o algún otro tipo de preajuste específico.
- [pattern](#) : Especifica una [expresión regular](#) que define un patrón que los datos que se introduzcan deben seguir.

Si los datos que se introducen en un campo de formulario siguen todas las reglas que especifican los atributos anteriores, se consideran válidos. Si no, se consideran no válidos.

Cuando un elemento es válido, se cumplen los aspectos siguientes:

- El elemento coincide con la pseudoclase `:valid` de CSS, lo que te permite aplicar un estilo específico a los elementos válidos.
- Si el usuario intenta enviar los datos, el navegador envía el formulario siempre que no haya nada más que lo impida (por ejemplo, JavaScript).

Cuando un elemento no es válido, se cumplen los aspectos siguientes:

- El elemento coincide con la pseudoclase `:invalid` de CSS, y a veces con otras pseudoclases de interfaz de usuario (UI) –por ejemplo, `:out-of-range` – dependiendo del error, que te permite aplicar un estilo específico a elementos no válidos.
- Si el usuario intenta enviar los datos, el navegador bloquea el formulario y muestra un mensaje de error.

7.6.1.1. A tener en cuenta...

Queremos que completar formularios web sea lo más fácil posible. Entonces, ¿por qué insistimos en validar nuestros formularios? Hay tres razones principales:

- **Queremos obtener los datos correctos en el formato correcto.** Nuestras aplicaciones no funcionarán correctamente si los datos de nuestros usuarios se almacenan en el formato incorrecto, son incorrectos o se omiten por completo.
- **Queremos proteger los datos de nuestros usuarios.** Obligar a nuestros usuarios a introducir contraseñas seguras facilita proteger la información de su cuenta.
- **Queremos protegernos a nosotros mismo.** Hay muchas formas en que los usuarios maliciosos puedan usar mal los formularios desprotegidos y dañar la aplicación (puedes

consultar [Seguridad del sitio web](#)).

Por todo ello debemos tener en cuenta que la validación en el lado del cliente *no debe considerarse* una medida de seguridad exhaustiva. Tus aplicaciones siempre deben realizar comprobaciones de seguridad de los datos enviados por el formulario *en el lado del servidor, así como también* en el lado del cliente, porque la validación en el lado del cliente es demasiado fácil de evitar, por lo que los usuarios malintencionados pueden enviar fácilmente datos incorrectos a tu servidor (puedes leer [Seguridad en los sitios web](#) para conocer más detalles).

Nosotros no entraremos en detalle ahora sobre cómo implementar la validación en el lado del servidor, ya que está fuera del alcance de esta unidad, pero debemos tenerlo en cuenta.

7.6.2. En el lado del servidor: Recuperando los datos

Sea cual sea el método HTTP que elijamos, el servidor recibe una cadena que será analizada con el fin de obtener los datos como una lista de pares clave/valor. La forma de acceder a esta lista depende de la plataforma de desarrollo que utilicemos y de las estructuras específicas que utilicemos.

[PHP](#) ofrece algunos objetos globales para acceder a los datos. Suponiendo que hayamos utilizado el método `GET`, el ejemplo que utilizaremos en las secciones siguientes cogerá los datos y los guardará en un fichero. Por supuesto, lo que se hace con los datos depende de nosotros. Es posible visualizarlos, almacenarlos en una base de datos, enviarlos por correo electrónico, o procesarlos de alguna otra manera.

Por motivos de sencillez y posibilidad de realizar pruebas en muchos entornos, utilizaremos PHP para completar nuestros ejemplos y conseguir que sean funcionales.

7.6.3. El método `GET`

El método `GET` es utilizado por el navegador para pedir al servidor que se envíe de vuelta un recurso dado: «Hey servidor, quiero conseguir este recurso.» En este caso, el navegador envía un cuerpo vacío. Debido a que el cuerpo está vacío, si un formulario se envía utilizando este método, los datos enviados al servidor se anexan a la URL.

Echando un vistazo a nuestro formulario de contacto, y teniendo en cuenta que hemos utilizado el método `GET`, cuando enviemos el formulario, veremos que los datos aparecen en la URL (en la barra de direcciones del navegador). Por ejemplo, si introducimos «Fernando» como nombre de usuario, «fernando@prueba.com» como dirección de correo

electrónico y «Hola» como mensaje, y pulsamos el botón «Enviar», deberíamos ver algo como esto en la barra de direcciones: «

`contactar.php?nombre=Fernando&correo=fernando@prueba.com&message=Hola` «.

Los datos se añaden a la URL como una serie de pares de nombre / valor. Después que la dirección web URL ha terminado, se incluye un signo de interrogación (?) seguido de los pares de nombre / valor, cada uno separado por un signo (&). En nuestro caso estamos enviando los siguientes datos al servidor:

- `nombre` , que tiene el valor «Fernando»
- `correo` , que tiene el valor «fernando@prueba.com»
- `mensaje` , que tiene el valor «Hola»

7.6.3.1. Ejercicio propuesto: Formulario de contactar

Crea una nueva página web con un formulario de contactar, utilizando el código del ejemplo anterior. Debería obtener algo similar al formulario que tienes a continuación. Comprueba el resultado en tu navegador y valida el código. También intente enviar los datos pulsando el botón y verifica la URL que aparece en la barra de direcciones. Finalmente, ajusta la longitud mínima y máxima de los campos de texto utilizando los valores que consideres adecuados para asegurarte de que los datos del formulario sean correctos antes de enviarlos al servidor.

— Ten en cuenta que si pulsas el botón enviar, irás a la página «contactar.php», que aún no está implementada. En este punto obtendrás un error, pero podrás ver toda la información en la URL, ya que estamos usando el método GET.

7.6.3.2. Ejercicio propuesto: Guardar datos de contacto

Continuemos con nuestro ejemplo PHP para guardar nuestros datos del formulario de contacto. Crea un archivo «contactar.php» con el siguiente código. Sube el formulario y el código PHP a tu servidor y prueba el ejemplo completo del formulario de contacto para verificar que los mensajes se van guardando en el servidor. Diles también a tus compañeros que prueben la página web y verifiquen que los datos que han introducido también están guardados.

```

2.    // La variable global $_GET nos permite acceder a los datos enviados con el
    método GET
3.    $nombre = $_GET['nombre'];
4.    $correo = $_GET['correo'];
5.    $mensaje = $_GET['mensaje'];
6.
7.    // Colocamos todos los datos en el archivo "mensajes.csv" en una nueva
    línea cada vez
8.    file_put_contents("mensajes.csv", "$nombre,$correo,$mensaje\n",
    FILE_APPEND);
9.
10.   // Mostramos un enlace a la página anterior y también al archivo para
    comprobar el resultado
11.   echo "<p>Datos guardados</p>";
12.   echo "<p>Haz click <a href='\"".$_SERVER['HTTP_REFERER']."'>aquí</a> para ir
    a la página anterior</p>";
13.   echo "<p>Haz click <a href='mensajes.csv' target='_blank'>aquí</a> para ver
    todos los mensajes</p>";
14.   ?>

```

7.6.3.3. Ejercicio propuesto: Formulario para saludar

Crea una nueva página web con un formulario similar al siguiente, comprobando el resultado en el navegador y validando el código. Pulsa el botón enviar y eche un vistazo a la barra de direcciones del navegador. Después de eso, introduce otros datos diferentes a los valores predeterminados, presione el botón Enviar de nuevo y verifique que la nueva URL contenga la información correcta. Finalmente, cambia el valor predeterminado de ambos campos de texto («Hola» y «Mamá») para usar otros valores y comprueba el resultado nuevamente.

— Ten en cuenta que si presionas el botón enviar, irá a la página «saludos.php», que aún no está implementada. En este punto obtendrás un error, pero verás toda la información en la URL, ya que estamos usando el método GET.

```

1.   <form action="saludos.php" method="GET">
2.       <p>
3.           <label>
4.               ¿Qué quieres decir para saludar?: <input name="saludo" value="Hola"
               required="" />
5.           </label>
6.       </p>
7.       <p>
8.           <label>
9.               ¿A quién se lo quieres decir?: <textarea name="persona"
               required="">Mamá</textarea>
10.          </label>
11.      </p>

```

```

12.     <p>
13.         <button>Enviar el saludo</button>
14.     </p>
15. </form>

```

7.6.3.4. Ejercicio propuesto: Guardar texto genérico

Continuemos con nuestro código PHP para guardar nuestros datos del formulario para saludar. Crea un archivo «saludar.php» con el código que aparece a continuación. Graba el formulario y el código PHP en el servidor y graba el ejemplo completo del formulario para comprobar que ahora los saludos ahora se guardan en el servidor. También diles a tus compañeros que prueben la página web y verifiquen que los datos que han introducido también estén guardados.

— Observarás las similitudes respecto al ejemplo anterior. Solo hemos cambiado las variables (\$saludo y \$persona) y el nombre del archivo donde se guardan los datos («saludos.csv»).

```

1.  <?php
2.      // La variable global $_GET nos permite acceder a los datos enviados con el
    método GET
3.      $saludo = $_GET['saludo'];
4.      $persona = $_GET['persona'];
5.
6.      // Colocamos todos los datos en el archivo "saludos.csv" en una nueva línea
    cada vez
7.      file_put_contents("saludos.csv", "$saludo,$persona\n", FILE_APPEND);
8.
9.      // Mostramos un enlace a la página anterior y también al archivo para
    comprobar el resultado
10.     echo "<p>Datos guardados</p>";
11.     echo "<p>Haz clic <a href='\"".$_SERVER['HTTP_REFERER']."'>aquí</a> para
    volver a la página anterior</p>";
12.     echo "<p>Haz clic <a href='saludos.csv' target='_blank'>aquí</a> para ver
    todos los saludos</p>";
13. ?>

```

7.6.4. El método POST

El método POST es un poco diferente. Es el método que el navegador utiliza para comunicarse con el servidor cuando se pide una respuesta que tenga en cuenta los datos proporcionados en el cuerpo de la petición HTTP: «Hey servidor, echa un vistazo a estos datos y envíame de vuelta un resultado apropiado.» Si un formulario se envía utilizando este método, los datos se anexan al cuerpo de la petición HTTP. Es más seguro que el

método GET, ya que cuando el formulario se envía mediante el método POST, ninguna otra persona que se encuentre cerca puede ver los datos. Este método se recomienda, por ejemplo, para ser utilizado en formularios donde se envía una contraseña.

Veamos el siguiente ejemplo, que es bastante similar al formulario en la sección GET anterior, pero con el atributo de método establecido en POST y el tipo de campo establecido en «password»:

```
1. <form action="login.php" method="POST">
2.   <p>
3.     <label>Usuario: <input type="text" name="usuario" required="" /></label>
4.   </p>
5.   <p>
6.     <label>Contraseña: <input type="password" name="contrasena" required=""
7.   /></label>
8.   </p>
9.   <p>
10.    <button>Comprobar usuario y contraseña</button>
11.  </p>
12. </form>
```

7.6.4.1. Ejercicio propuesto: Formulario de login

Crea una nueva página web con un formulario de inicio de sesión, utilizando el código del ejemplo anterior. Deberías obtener un resultado similar al que se muestra a continuación. Comprueba el resultado en tu navegador y valida el código. A continuación intenta enviar los datos pulsando el botón y observa si hay alguna información en la URL. Finalmente, establece la longitud mínima del campo de texto del usuario en 5 y la máxima en 10, y haz lo mismo para el campo de contraseña.

— Ten en cuenta que si pulsas el botón enviar, irás a la página «login.php», que aún no está implementada. En este punto obtendrás un error, pero no verás ninguna información en la URL, ya que estamos usando el método POST.

7.6.4.2. Ejercicio propuesto: Comprobar datos privados

Continuemos con nuestro código PHP para comprobar el usuario y la contraseña del formulario de inicio de sesión. Crea un archivo «login.php» con el siguiente código. Sube el formulario y el código PHP a tu servidor y prueba tu ejemplo completo del formulario de inicio de sesión para comprobar el usuario («admin») y la contraseña («1234»). También dile a tus compañeros que prueben tu página web. Después de eso, cambia la

contraseña del archivo «login.php» y pide a tus compañeros que intenten adivinar la nueva contraseña. Debes usar una contraseña muy simple de entre las que aparecen en «https://en.wikipedia.org/wiki/List_of_the_most_common_passwords» (de lo contrario, es posible que tus compañeros no puedan adivinarla).

— Observarás las similitudes respecto al ejemplo anterior. Solo hemos cambiado las variables (\$usuario y \$contrasena) y hemos utilizado una condición para mostrar una imagen con el pulgar hacia arriba o hacia abajo, dependiendo de si la contraseña es correcta o no.

```
1.  <?php
2.      // La variable global $_POST nos permite acceder a los datos enviados con el
    método POST
3.      $usuario = $_POST['usuario'];
4.      $contrasena = $_POST['contrasena'];
5.
6.      // Comprueba el usuario y la contraseña
7.      if ($usuario == "admin" && $contrasena == "1234") {
8.          echo "<img
    src='https://raw.githubusercontent.com/twbs/icons/main/icons/hand-thumbs-
    up.svg' width='100' />";
9.          echo "<p>¡Perfecto! :-)</p><p>Haz clic <a
    href='\".$_SERVER['HTTP_REFERER'].\">aquí</a> para ir a la página
    anterior</p>";
10.     }
11.     else {
12.         echo "<img
    src='https://raw.githubusercontent.com/twbs/icons/main/icons/hand-thumbs-
    down.svg' width='100' />";
13.         echo "<p>¡Usuario o contraseña incorrectos! :-(</p><p>Haz clic <a
    href='\".$_SERVER['HTTP_REFERER'].\">aquí</a> para probar de nuevo</p>";
14.     }
15.  ?>
```

8. Test

Comprueba tus conocimientos con este [test](#) sobre formularios y otros conceptos relacionados con esta unidad.